

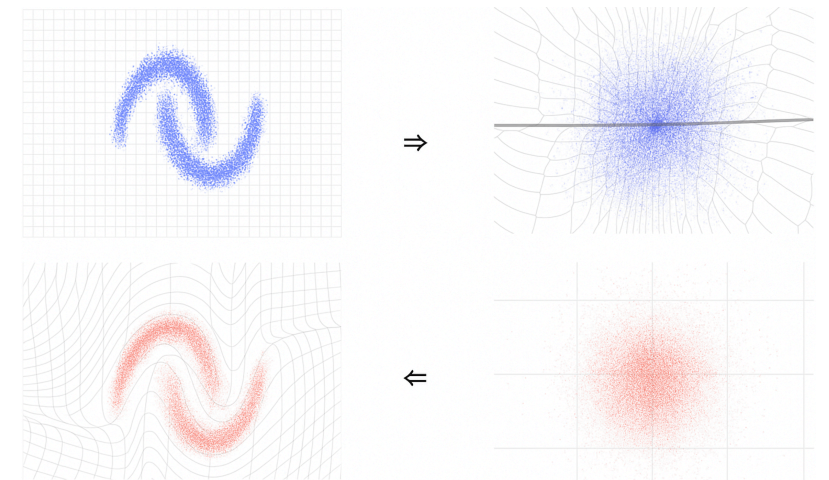
Generative Deep Learning

Normalizing Flows

Alejandro Lancho Serrano, alancho@ing.uc3m.es

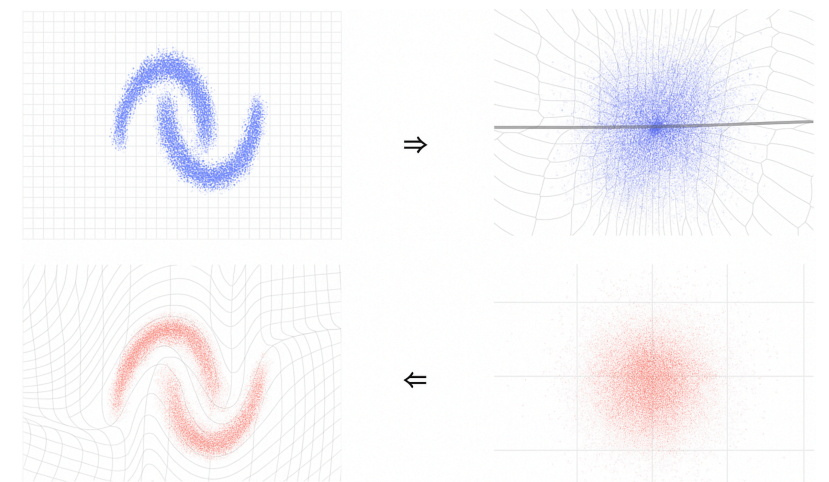
Why Study Normalizing Flows?

- Learn **exact probabilistic models** over high-dimensional data
- Use **invertible neural networks** for lossless sampling
- Enable **exact log-likelihood computation** — unlike VAEs or GANs



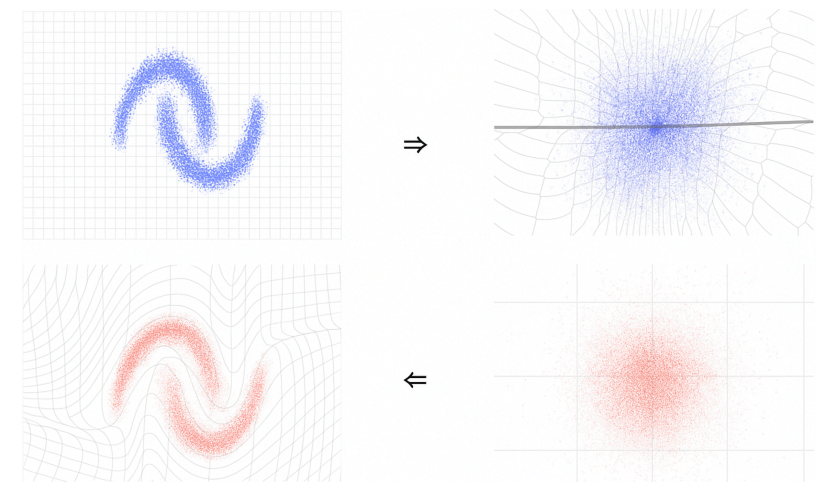
Why Study Normalizing Flows?

- Learn **exact probabilistic models** over high-dimensional data
- Use **invertible neural networks** for lossless sampling
- Enable **exact log-likelihood computation** — unlike VAEs or GANs
- Useful for:
 - ▶ Generative modeling (images, audio, text)
 - ▶ Density estimation
 - ▶ Representation learning
 - ▶ Scientific modeling (e.g., inverse problems, physics-informed ML)



Why Study Normalizing Flows?

- Learn **exact probabilistic models** over high-dimensional data
- Use **invertible neural networks** for lossless sampling
- Enable **exact log-likelihood computation** — unlike VAEs or GANs
- Useful for:
 - ▶ Generative modeling (images, audio, text)
 - ▶ Density estimation
 - ▶ Representation learning
 - ▶ Scientific modeling (e.g., inverse problems, physics-informed ML)



What if we could map a complex distribution into a simple one (e.g., a Gaussian), and go back and forth perfectly?

What Do We Want from a Generative Model?

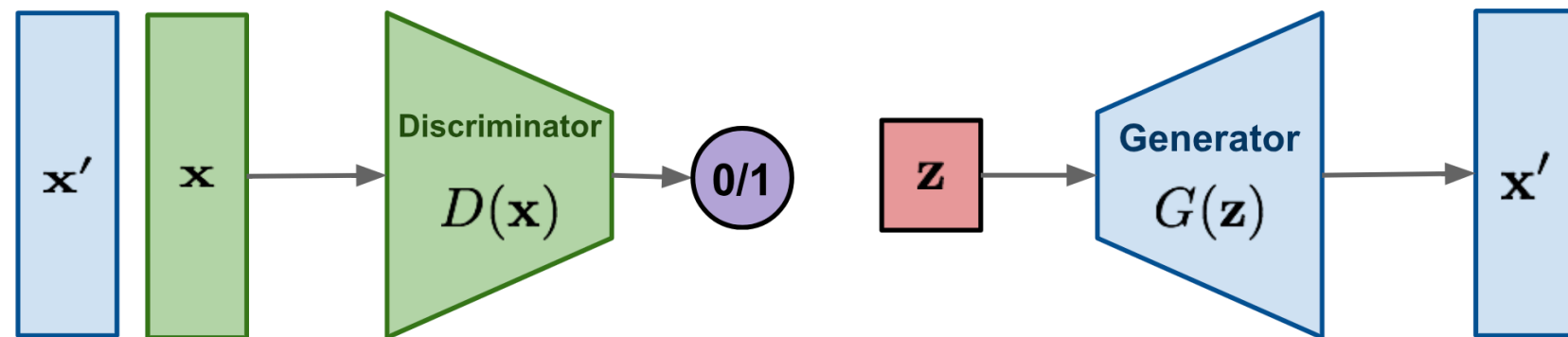
- **Sample** realistic data points → generate new digits, images, text...
- **Compute likelihoods** → How probable is a given input under the model?
- **Train efficiently** → Optimizable via gradient-based methods
- **Support inference and uncertainty** → What is the uncertainty in a prediction?

What Do We Want from a Generative Model?

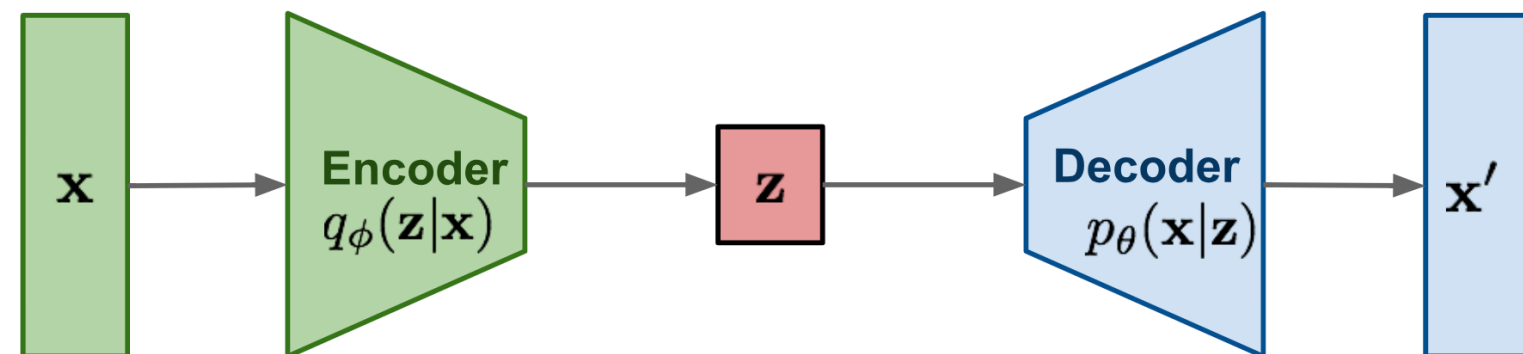
- **Sample** realistic data points → generate new digits, images, text...
- **Compute likelihoods** → How probable is a given input under the model?
- **Train efficiently** → Optimizable via gradient-based methods
- **Support inference and uncertainty** → What is the uncertainty in a prediction?
- Common approaches:
 - ▶ **GANs:** ✓ **Sampling** | ✗ **No likelihood** | ✗ **Hard to train** (mode collapse, instability)
 - ▶ **VAEs:** ✓ **Sampling** | ✓ **Approximate likelihood** (ELBO)
 - ✓ **Latent representations** | ✗ **Approximate inference**
 - ▶ **Normalizing Flows:** ✓ **Exact sampling** | ✓ **Exact likelihood**
 - ✓ **Invertible latent representations**
 - ✓ **Tractable, expressive, and interpretable!**

A Visual Comparison among Generative Models

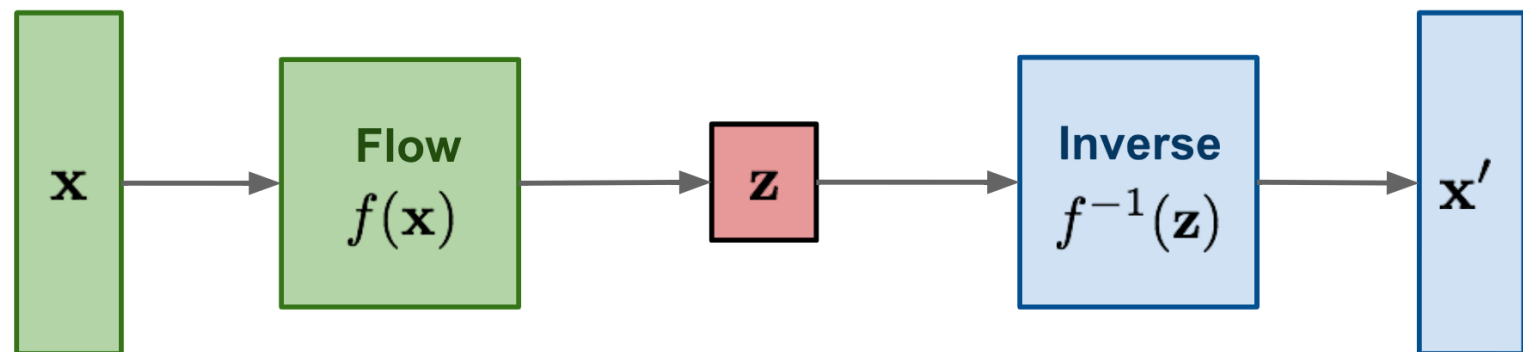
GAN: minimax the classification error loss.



VAE: maximize ELBO.



Flow-based generative models: minimize the negative log-likelihood



Core Ingredients of Normalizing Flows (I)

- **Invertible Transformations**

- ▶ A bijective function $f : \mathbb{R}^D \rightarrow \mathbb{R}^D$
- ▶ Allows both **sampling** and **density evaluation**
- ▶ Enables transforming a complex distribution x into a simple latent $z \sim p(z)$, and vice versa

$$z = f(x), \quad x = f^{-1}(z)$$

Core Ingredients of Normalizing Flows (I)

- **Invertible Transformations**

- ▶ A bijective function $f : \mathbb{R}^D \rightarrow \mathbb{R}^D$
- ▶ Allows both **sampling** and **density evaluation**
- ▶ Enables transforming a complex distribution x into a simple latent $z \sim p(z)$, and vice versa

$$z = f(x), \quad x = f^{-1}(z)$$

- **Change of Variables Formula:** Given a latent distribution $p(z)$, the likelihood of x is:

$$p(x) = p(z) \cdot \left| \det \left(\frac{\partial f(x)}{\partial x} \right) \right|$$

Or in log-space (more stable):

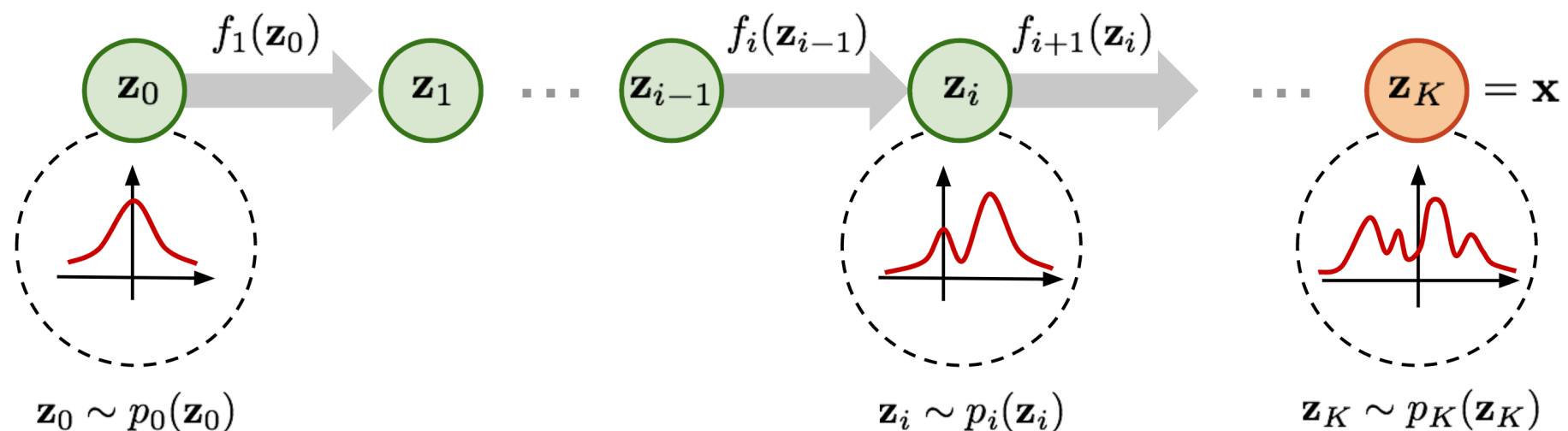
$$\log p(x) = \log p(z) + \log \left| \det \left(\frac{\partial f(x)}{\partial x} \right) \right|$$

Core Ingredients of Normalizing Flows (II)

● Maximum Likelihood Training

- ▶ Optimize parameters θ to **maximize log-likelihood** of observed data
- ▶ Unlike GANs or VAEs, flows provide **exact log-likelihood**
- ▶ Trained via gradient descent on:

$$\max_{\theta} \mathbb{E}_{x \sim \text{data}} [\log p_{\theta}(x)]$$



Change of Variable: The Core Equation

To compute the density of a sample after a transformation → **change-of-variable formula**

- **1D case:**

$$p_x(x) = p_z(f(x)) \cdot \left| \frac{df(x)}{dx} \right|$$

- **Higher Dimensions:**

$$\log p_X(\mathbf{x}) = \log p_Z(f(\mathbf{x})) + \log \left| \det \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right|$$

Change of Variable: The Core Equation

To compute the density of a sample after a transformation → **change-of-variable formula**

- **1D case:**

$$p_x(x) = p_z(f(x)) \cdot \left| \frac{df(x)}{dx} \right|$$

- **Higher Dimensions:**

$$\log p_X(\mathbf{x}) = \log p_Z(f(\mathbf{x})) + \log \left| \det \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right|$$

- Key concepts:

- ▶ f : invertible transformation from data space to latent space
- ▶ p_Z : simple base density (e.g., Gaussian)
- ▶ Jacobian determinant: measures how volume is stretched or compressed

Change of Variable: The Core Equation

To compute the density of a sample after a transformation → **change-of-variable formula**

- **1D case:**

$$p_x(x) = p_z(f(x)) \cdot \left| \frac{df(x)}{dx} \right|$$

- **Higher Dimensions:**

$$\log p_X(\mathbf{x}) = \log p_Z(f(\mathbf{x})) + \log \left| \det \frac{\partial f(\mathbf{x})}{\partial \mathbf{x}} \right|$$

- Key concepts:

- ▶ f : invertible transformation from data space to latent space
- ▶ p_Z : simple base density (e.g., Gaussian)
- ▶ Jacobian determinant: measures how volume is stretched or compressed

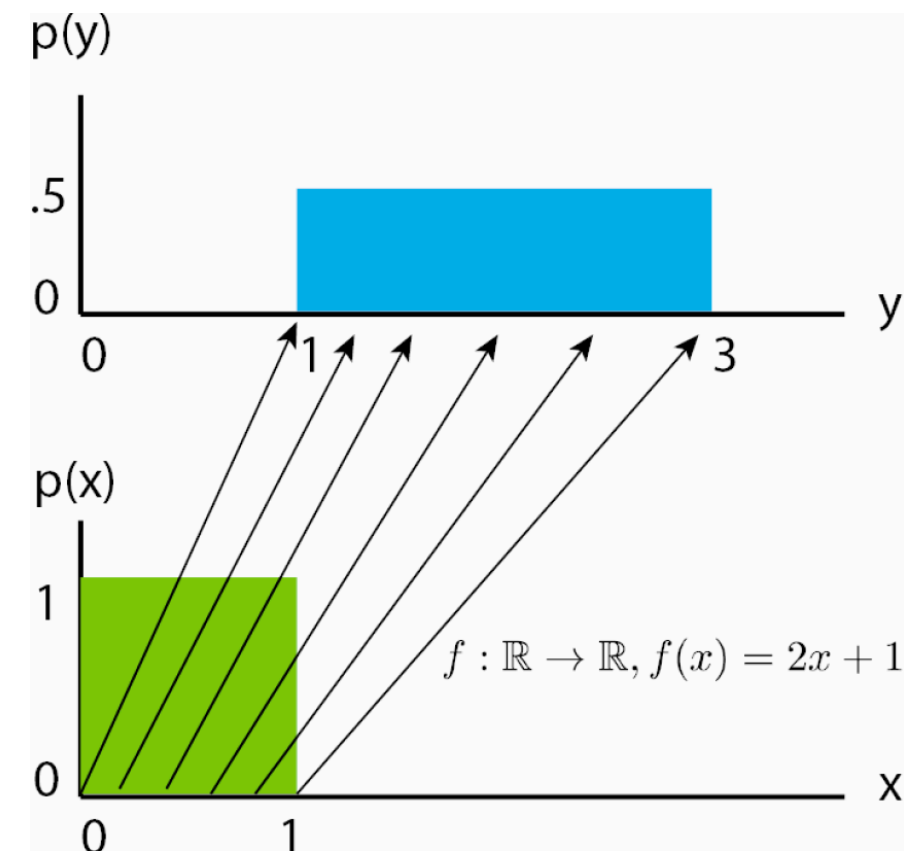
**This formula is the foundation of normalizing flows:
It lets us compute exact log-likelihoods!**

A Simple Example: Linear Flow in 1D

$$y = f(x) = ax + b \quad \text{with } a \neq 0$$

Then:

$$p_X(x) = p_Y(f(x)) \cdot |a| \quad \Rightarrow \quad \log p_X(x) = \log p_Y(f(x)) + \log |a|$$



A Simple Example: Linear Flow in 1D

$$y = f(x) = ax + b \quad \text{with } a \neq 0$$

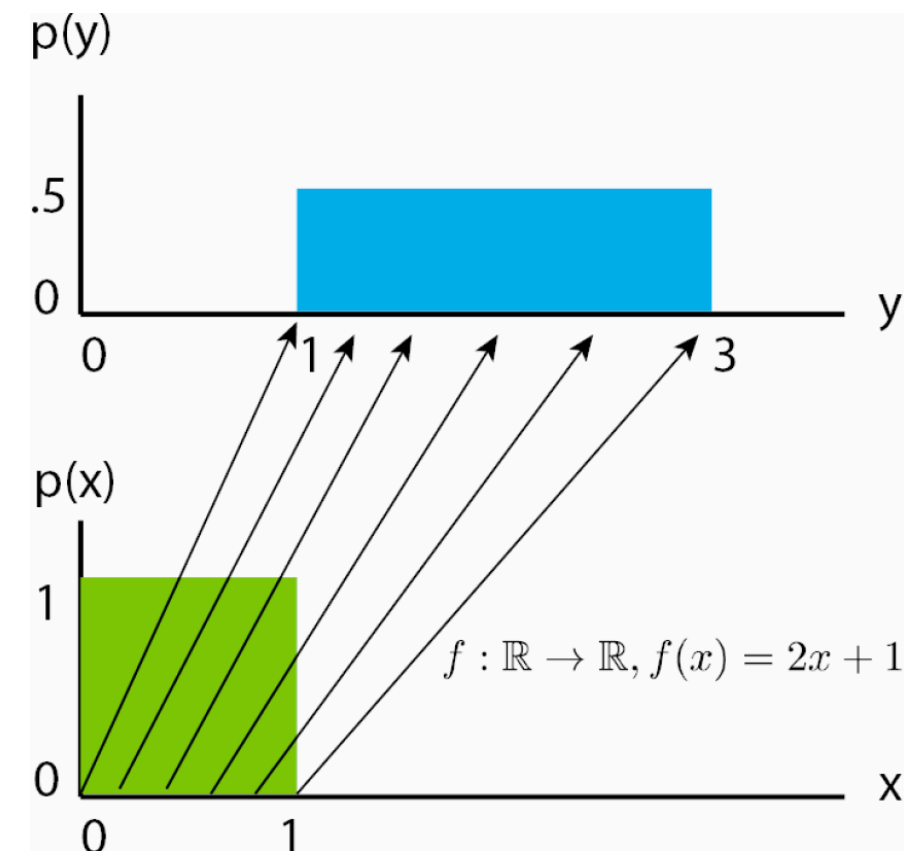
Then:

$$p_X(x) = p_Y(f(x)) \cdot |a| \quad \Rightarrow \quad \log p_X(x) = \log p_Y(f(x)) + \log |a|$$

● Assume:

- ▶ $x \sim \text{Unif}[0,1]$, $a = 2, b = 1 \rightarrow y = f(x) = 2x + 1$
- ▶ The transformation maps $x \in [0,1]$ to $y \in [1,3]$
- ▶ Probability mass must be preserved:

$$p_Y(y) = p_X(f^{-1}(y)) \cdot \left| \frac{d}{dy} f^{-1}(y) \right| = 1 \cdot \left| \frac{1}{2} \right| = \frac{1}{2} \quad \text{for } y \in [1,3]$$



Change of Variable in Higher Dimensions

- **To model a complex data distribution $p_X(x)$,**
 - ➡ start from a simple base distribution $p_Z(z)$ (e.g., standard Gaussian)
 - ➡ apply an invertible transformation $x = f(z)$

Change of Variable in Higher Dimensions

- **To model a complex data distribution $p_X(x)$,**
 - ➡ start from a simple base distribution $p_Z(z)$ (e.g., standard Gaussian)
 - ➡ apply an invertible transformation $x = f(z)$

- **The Change of Variables Formula:**

▶ If $f : \mathbb{R}^D \rightarrow \mathbb{R}^D$ is bijective and differentiable, using $z = f^{-1}(x)$,

$$\log p_X(x) = \log p_Z(z) + \log \left| \det \left(\frac{\partial f^{-1}(x)}{\partial x} \right) \right|$$

Change of Variable in Higher Dimensions

- To model a complex data distribution $p_X(x)$,

- ➡ start from a simple base distribution $p_Z(z)$ (e.g., standard Gaussian)

- ➡ apply an invertible transformation $x = f(z)$

- The Change of Variables Formula:

- ▶ If $f : \mathbb{R}^D \rightarrow \mathbb{R}^D$ is bijective and differentiable, using $z = f^{-1}(x)$,

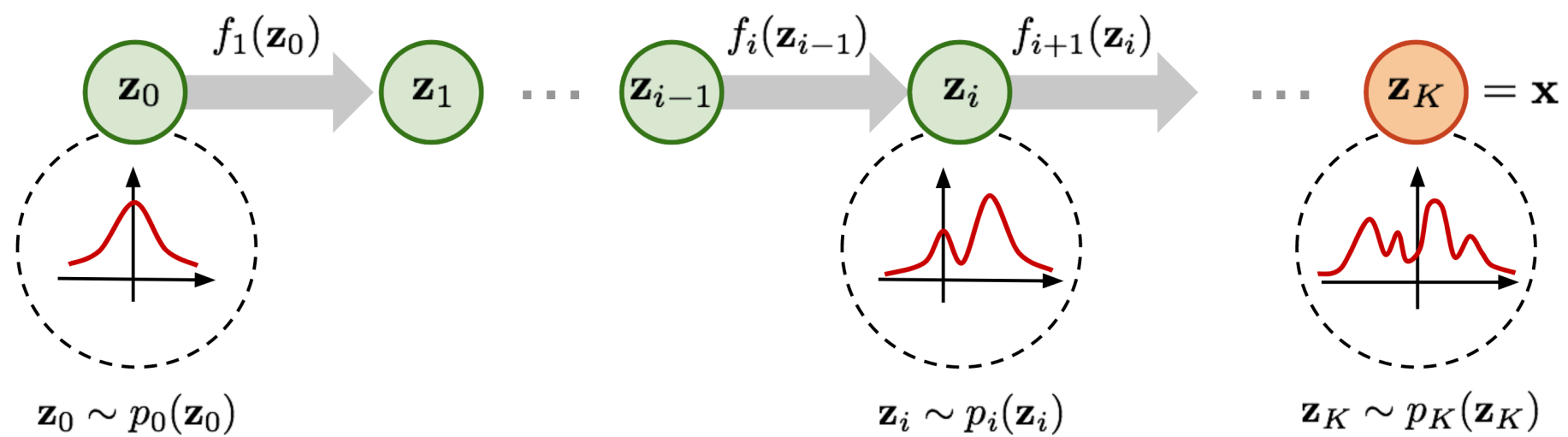
$$\log p_X(x) = \log p_Z(z) + \log \left| \det \left(\frac{\partial f^{-1}(x)}{\partial x} \right) \right|$$

- Interpretation:

- ▶ The **first term** measures how likely the latent code z is under the base distribution

- ▶ The **second term** corrects for how much the transformation stretches or squashes volume locally

Change of Variable in Higher Dimensions in Practice



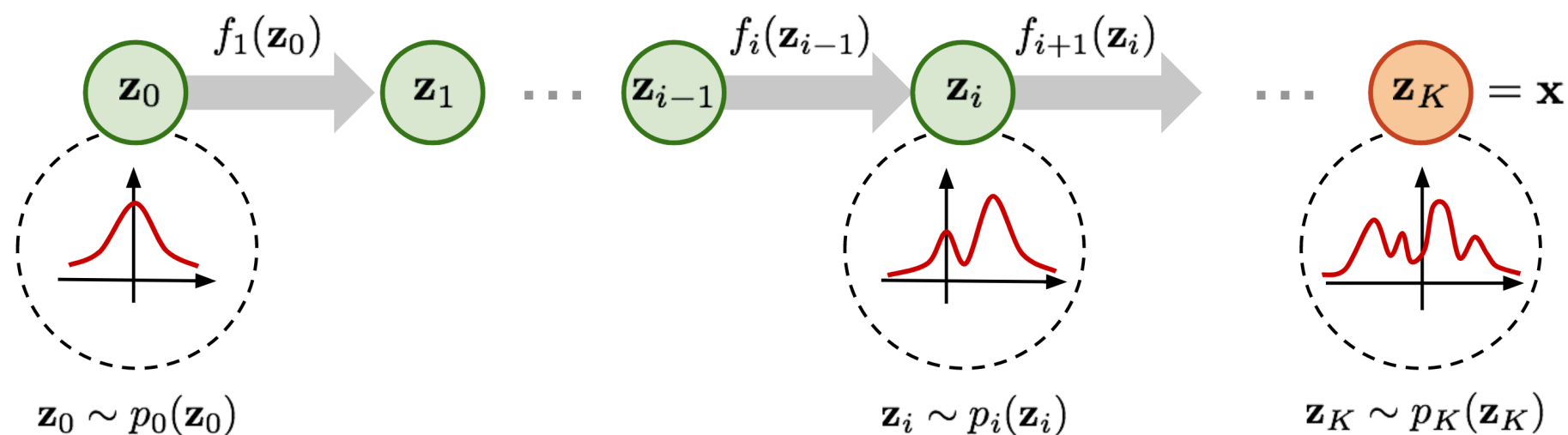
Change of Variable in Higher Dimensions in Practice

- We use a sequence of transformations:

$$x = f_K \circ f_{K-1} \circ \cdots \circ f_1(z)$$

- And sum the log-determinant terms:

$$\log p_X(x) = \log p_Z(z) + \sum_{k=1}^K \log \left| \det \left(\frac{\partial f_k^{-1}}{\partial x_k} \right) \right|$$



Designing Effective Flow Layers (I)

Designing Effective Flow Layers (I)

To build a powerful generative model, each transformation layer f_k must satisfy key properties:

1) Invertibility:

- Each layer must be **bijective** so we can:
 - ▶ **sample**: $x = f(z)$
 - ▶ **evaluate likelihoods**: $z = f^{-1}(x)$

Designing Effective Flow Layers (I)

To build a powerful generative model, each transformation layer f_k must satisfy key properties:

1) Invertibility:

- Each layer must be **bijective** so we can:
 - ▶ **sample**: $x = f(z)$
 - ▶ **evaluate likelihoods**: $z = f^{-1}(x)$

2) Efficient Jacobian Determinant:

- We need to compute:

$$\log \left| \det \left(\frac{\partial f_k^{-1}}{\partial x_k} \right) \right|$$

- This must be **tractable** (linear or better), or else training becomes impractical

Designing Effective Flow Layers (II)

3) Expressiveness:

- The transformation should be flexible enough to model complex dependencies and reshape densities
- But it must **balance expressiveness with computational cost**

Designing Effective Flow Layers (II)

3) Expressiveness:

- The transformation should be flexible enough to model complex dependencies and reshape densities
- But it must **balance expressiveness with computational cost**

Trade-Off in Flow Design:

Property	Simple Layer (e.g., Affine)	Complex Layer (e.g., Neural Spline)
Invertible	✓	✓
Tractable Jacobian	✓	✓ (with design care)
Expressiveness	✗	✓✓✓

Designing Effective Flow Layers (II)

3) Expressiveness:

- The transformation should be flexible enough to model complex dependencies and reshape densities
- But it must **balance expressiveness with computational cost**

Trade-Off in Flow Design:

Property	Simple Layer (e.g., Affine)	Complex Layer (e.g., Neural Spline)
Invertible	✓	✓
Tractable Jacobian	✓	✓ (with design care)
Expressiveness	✗	✓✓✓

Common Solution: Stack Many Simple Layers

- Chain multiple simple invertible transformations to build expressive, trainable flows.

Affine Coupling Layers

Affine Coupling Layers

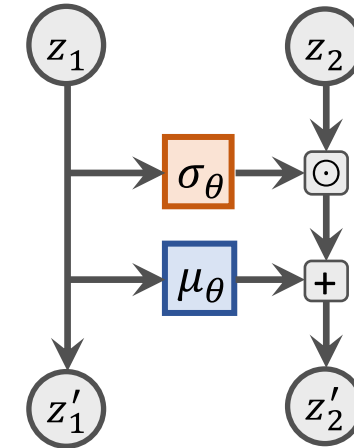
- **Core idea:** Split input into two parts $\rightarrow$ transform one, keep the other
- Guarantees invertibility and tractable Jacobian

Affine Coupling Layers

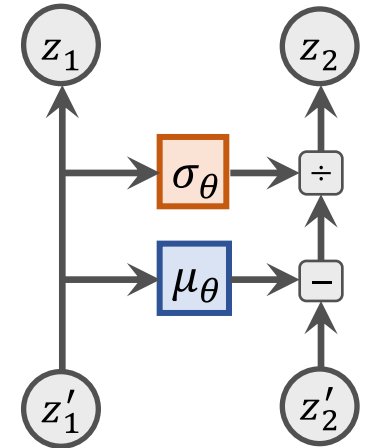
- **Core idea:** Split input into two parts $\rightarrow$ transform one, keep the other
- Guarantees invertibility and tractable Jacobian

Given $z = (z_{1:j}, z_{j+1:d})$, define:

- $z'_{1:j} = z_{1:j}$ (unchanged) and $z'_{j+1:d} = \mu_{\theta}(z_{1:j}) + \sigma_{\theta}(z_{1:j}) \odot z_{j+1:d}$



Forward



Inverse

Affine Coupling Layers

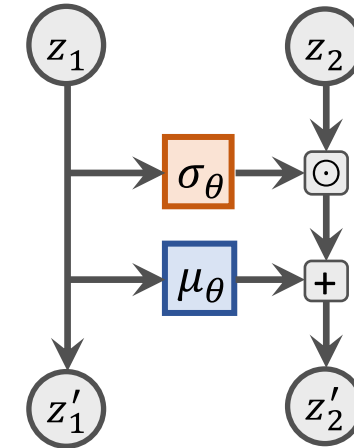
- **Core idea:** Split input into two parts $\rightarrow$ transform one, keep the other
- Guarantees invertibility and tractable Jacobian

Given $z = (z_{1:j}, z_{j+1:d})$, define:

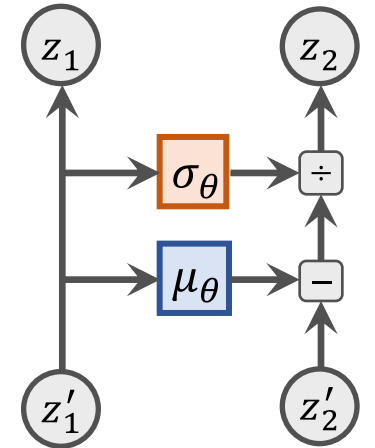
- $z'_{1:j} = z_{1:j}$ (unchanged) and $z'_{j+1:d} = \mu_{\theta}(z_{1:j}) + \sigma_{\theta}(z_{1:j}) \odot z_{j+1:d}$

Notation:

- $(\mu_{\theta}, \sigma_{\theta})$: outputs of a neural network
- $\odot$: element-wise multiplication



Forward



Inverse

Affine Coupling Layers

- **Core idea:** Split input into two parts $\rightarrow$ transform one, keep the other
- Guarantees invertibility and tractable Jacobian

Given $z = (z_{1:j}, z_{j+1:d})$, define:

- $z'_{1:j} = z_{1:j}$ (unchanged) and $z'_{j+1:d} = \mu_{\theta}(z_{1:j}) + \sigma_{\theta}(z_{1:j}) \odot z_{j+1:d}$

Notation:

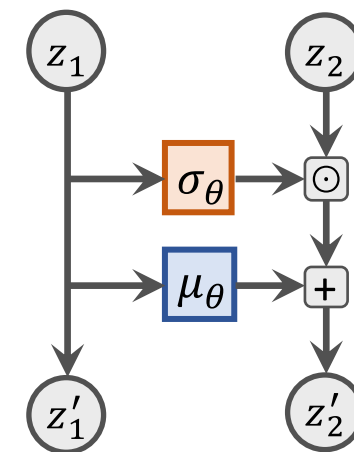
- $(\mu_{\theta}, \sigma_{\theta})$: outputs of a neural network
- $\odot$: element-wise multiplication

Why this design works?

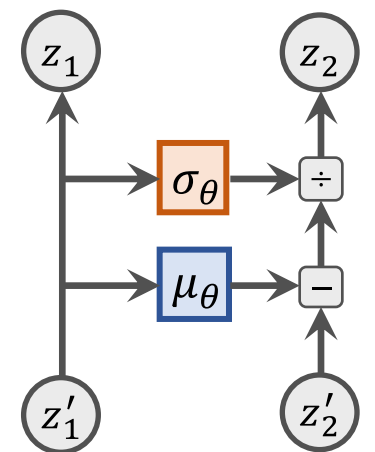
- Easy to compute inverse: $z_{j+1:d} = \frac{z'_{j+1:d} - \mu_{\theta}(z_{1:j})}{\sigma_{\theta}(z_{1:j})}$

- Efficient Jacobian determinant:

$$\log |\det J| = \sum_i \log \sigma_{\theta,i}(z_{1:j})$$



Forward



Inverse

Computing Likelihood via the Change of Variable Trick

Computing Likelihood via the Change of Variable Trick

- To compute the likelihood of $\mathbf{x}$, we map it to a latent $\mathbf{z}$ via an invertible function $\mathbf{z} = f(\mathbf{x})$
- The log-likelihood is given by:

$$\log p_X(\mathbf{x}) = \log p_Z(f(\mathbf{x})) + \log \left| \det \frac{df(\mathbf{x})}{d\mathbf{x}} \right|$$

Computing Likelihood via the Change of Variable Trick

- To compute the likelihood of $\mathbf{x}$, we map it to a latent $\mathbf{z}$ via an invertible function $\mathbf{z} = f(\mathbf{x})$
- The log-likelihood is given by:

$$\log p_X(\mathbf{x}) = \log p_Z(f(\mathbf{x})) + \log \left| \det \frac{df(\mathbf{x})}{d\mathbf{x}} \right|$$

- **Intuition reminder:**

- ▶ The **first term** measures how likely the latent $\mathbf{z}$ is under the prior
- ▶ The **second term** accounts for the **volume change** induced by the transformation.

Computing Likelihood via the Change of Variable Trick

- To compute the likelihood of $\mathbf{x}$, we map it to a latent $\mathbf{z}$ via an invertible function $\mathbf{z} = f(\mathbf{x})$
- The log-likelihood is given by:

$$\log p_X(\mathbf{x}) = \log p_Z(f(\mathbf{x})) + \log \left| \det \frac{df(\mathbf{x})}{d\mathbf{x}} \right|$$

- **Intuition reminder:**

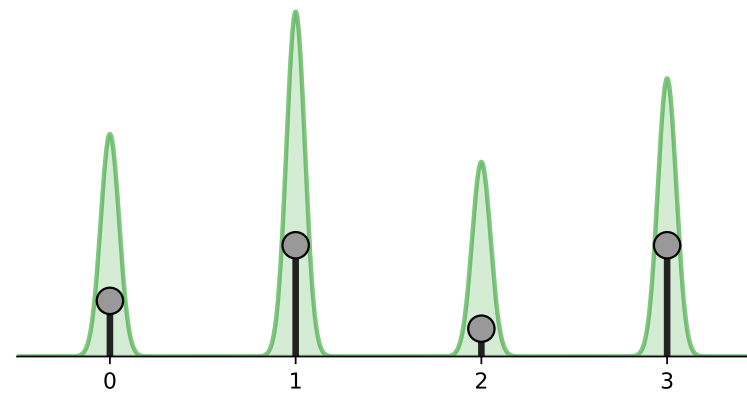
- ▶ The **first term** measures how likely the latent $\mathbf{z}$ is under the prior
- ▶ The **second term** accounts for the **volume change** induced by the transformation.

Both terms are tractable in flows, which enables exact log-likelihood

This is one of the main advantages of flows over VAEs or GANs!

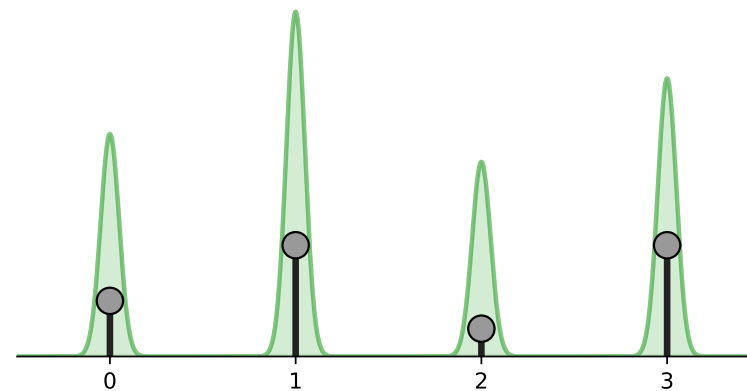
Dequantization

- Normalizing flows have **problems to deal with discrete data!**
 - ▶ Normalizing flows model continuous distributions — images discrete pixel values
 - ▶ Without dequantization, flows can overfit: **infinite density to exact pixel values**



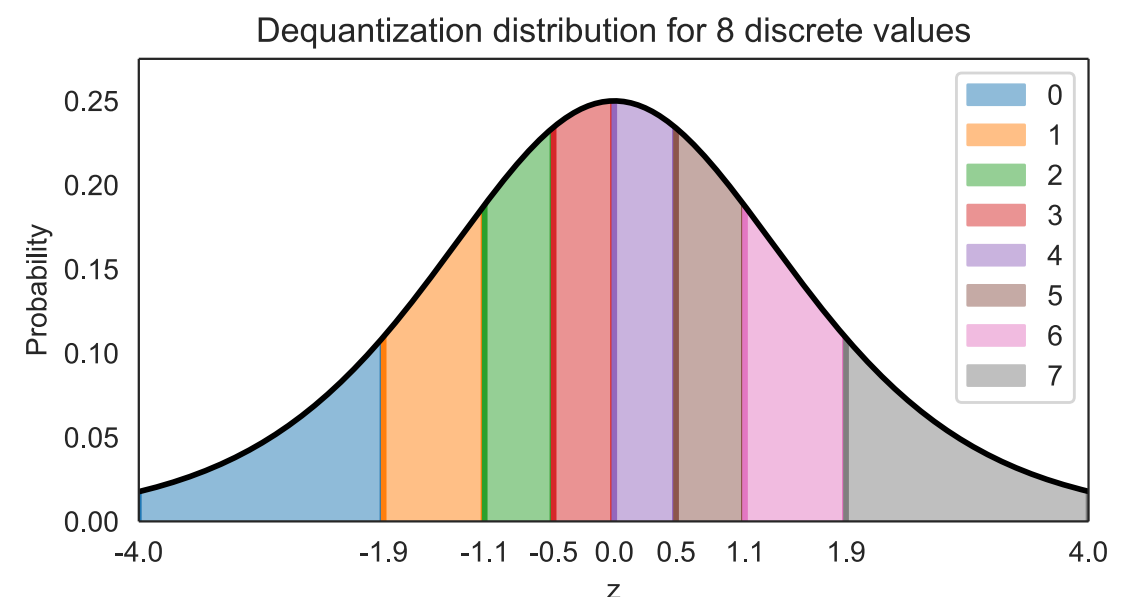
Dequantization

- Normalizing flows have **problems to deal with discrete data!**
 - ▶ Normalizing flows model continuous distributions — images discrete pixel values
 - ▶ Without dequantization, flows can overfit: **infinite density to exact pixel values**



- **Solution: Uniform Dequantization**

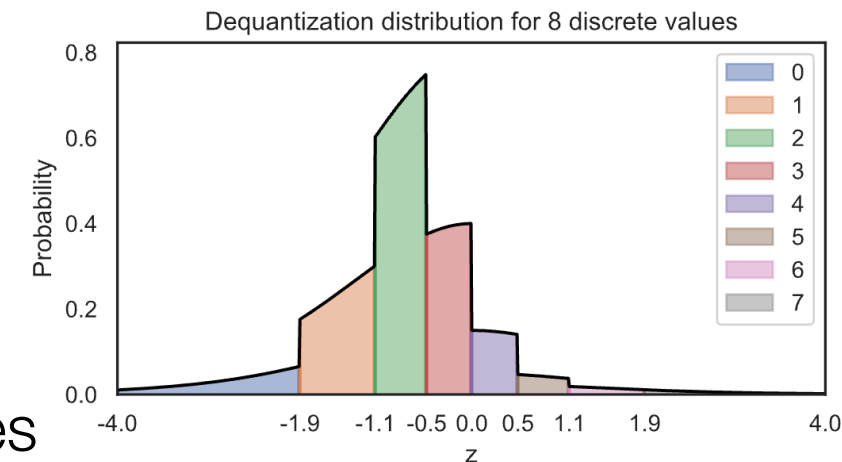
1. Add uniform noise: $u \sim \text{Unif}[0,1)^D$
2. Dequantize each pixel: $v = x + u$
3. Scale to $[0, 1]$: $v = \frac{x + u}{256}$
4. Map to $\mathbb{R}$ with inverse sigmoid (logit):
 - ▶ Ensures input is unbounded, matching the domain of Gaussian priors.



Is Uniform Dequantization Enough?

● Intuition:

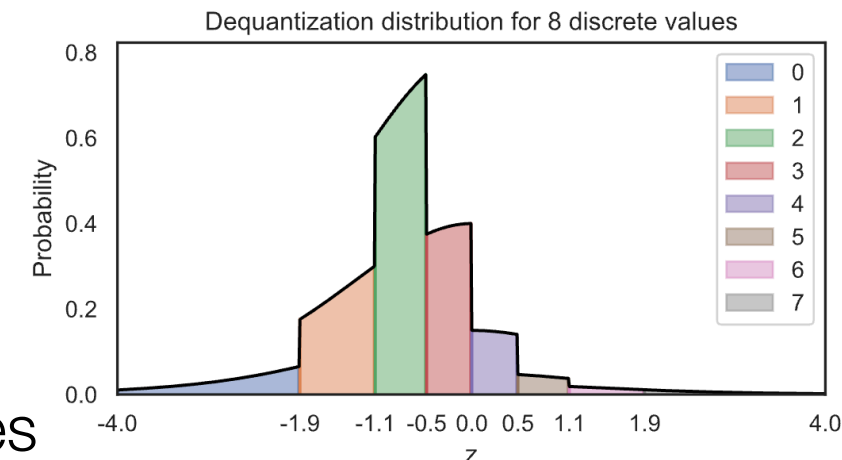
- ▶ The uniform noise breaks ties between integer values
- ▶ Avoids degenerate spikes in the learned density
- ▶ Allows gradients to flow through all pixel values
- ▶ Enables valid likelihood computation via change of variables



Is Uniform Dequantization Enough?

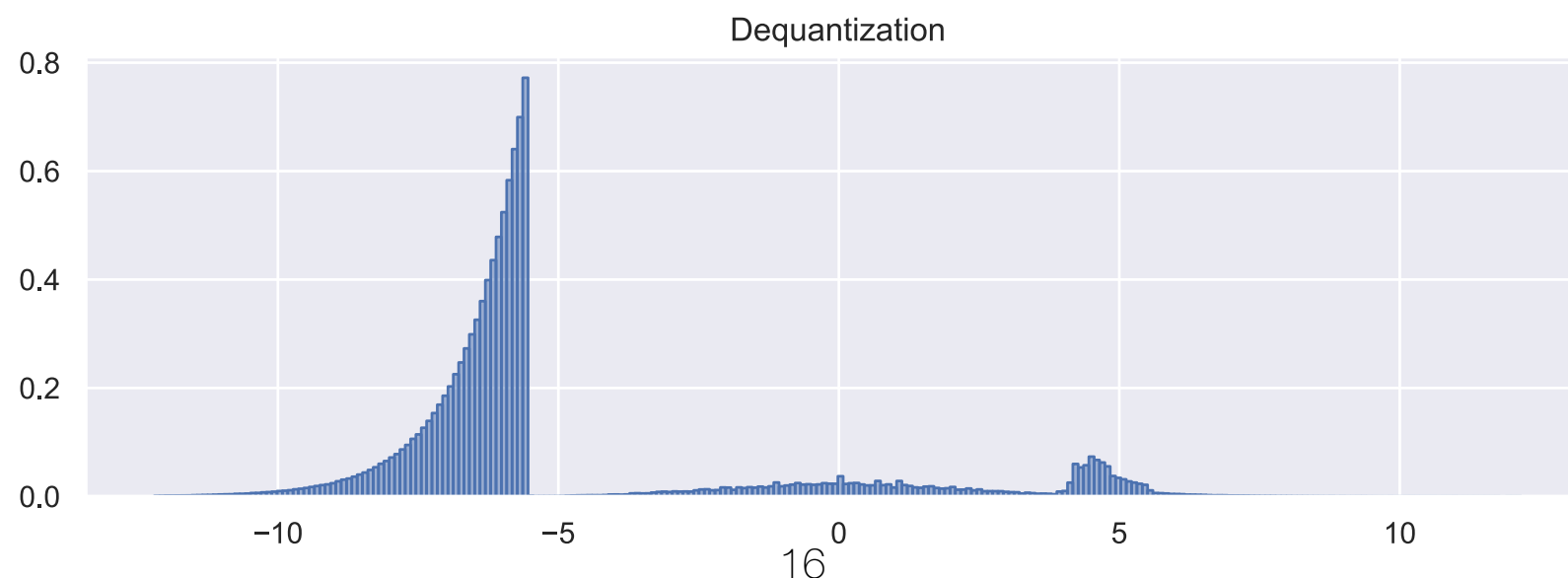
● Intuition:

- ▶ The uniform noise breaks ties between integer values
- ▶ Avoids degenerate spikes in the learned density
- ▶ Allows gradients to flow through all pixel values
- ▶ Enables valid likelihood computation via change of variables



● Limitations:

- ▶ The distribution after dequantization is **piecewise constant** and **non-smooth**
- ▶ Sharp, axis-aligned boundaries (hypercubes) appear in input space
- ▶ Frequent pixel values (like 0 in MNIST) lead to density spikes — hard for flows to model
- ▶ Ignores structure in the data — all pixels treated equally



Variational Dequantization: A Smarter Way to Bridge Discreteness

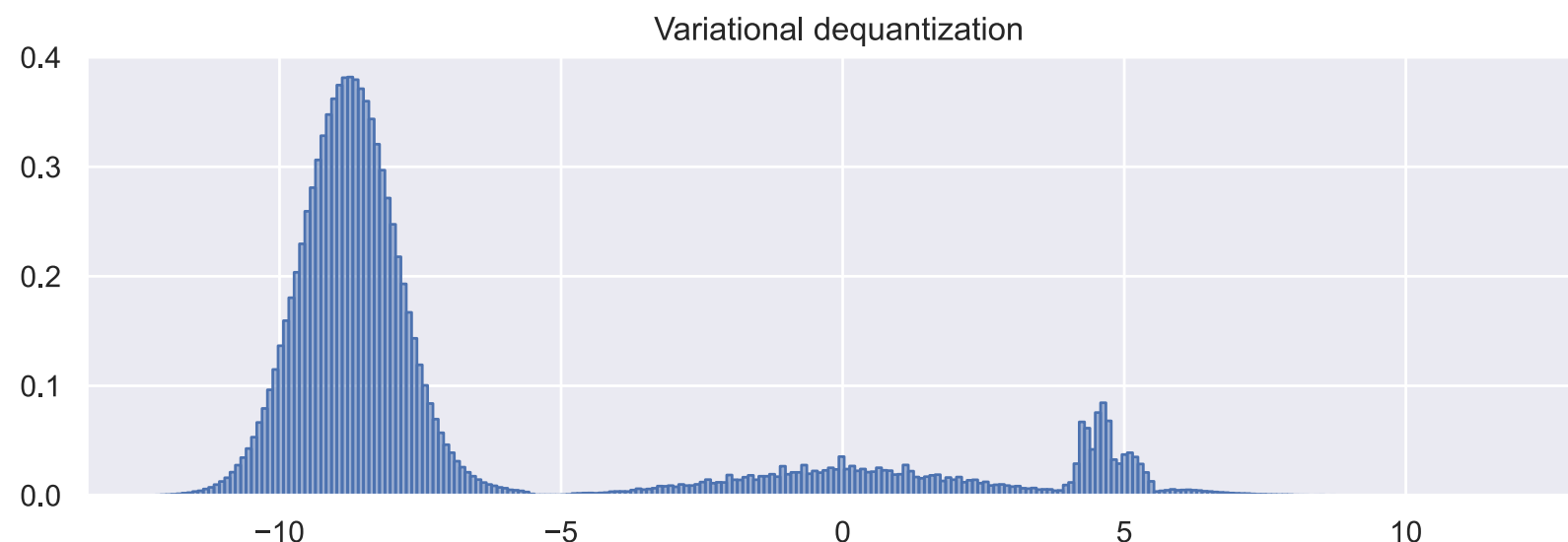
- We make the **noise learnable via a conditional distribution** $q(u | x)$:
 - ▶ Use a small normalizing flow, conditioned on image x
 - ▶ Samples lie in $[0,1)^D$, shaped to match local pixel statistics
 - ▶ Gives the model **flexibility to dequantize** differently for background vs. foreground pixels

Variational Dequantization: A Smarter Way to Bridge Discreteness

- We make the **noise learnable via a conditional distribution** $q(u | x)$:
 - ▶ Use a small normalizing flow, conditioned on image x
 - ▶ Samples lie in $[0,1)^D$, shaped to match local pixel statistics
 - ▶ Gives the model **flexibility to dequantize** differently for background vs. foreground pixels
- **Intuition:**
 - ▶ Variational dequantization **adapts the noise to the image content**
 - ➡ easier for the main flow to model global structure
 - ▶ Learns to “reshape” the discrete image into a smooth, flow-friendly input
 - ▶ Encourages better likelihoods, faster convergence, and improved generalization

Variational Dequantization: A Smarter Way to Bridge Discreteness

- We make the **noise learnable via a conditional distribution** $q(u | x)$:
 - ▶ Use a small normalizing flow, conditioned on image x
 - ▶ Samples lie in $[0,1)^D$, shaped to match local pixel statistics
 - ▶ Gives the model **flexibility to dequantize** differently for background vs. foreground pixels
- **Intuition:**
 - ▶ Variational dequantization **adapts the noise to the image content**
 - ➡ easier for the main flow to model global structure
 - ▶ Learns to “reshape” the discrete image into a smooth, flow-friendly input
 - ▶ Encourages better likelihoods, faster convergence, and improved generalization



Training Normalizing Flows

Training Normalizing Flows

- **Goal:** Maximize the likelihood of the data under the model
- **Loss function:** Negative log-likelihood (to be minimized)

$$\mathcal{L}(x) = -\log p(x) = -\log p(z) - \log \left| \det \left(\frac{\partial f}{\partial x} \right) \right|$$

Training Normalizing Flows

- **Goal:** Maximize the likelihood of the data under the model
- **Loss function:** Negative log-likelihood (to be minimized)

$$\mathcal{L}(x) = -\log p(x) = -\log p(z) - \log \left| \det \left(\frac{\partial f}{\partial x} \right) \right|$$

- ▶ $z = f(x)$: output of the flow
- ▶ $p(z)$: usually a standard normal
- ▶ The Jacobian term is computed efficiently via coupling layers

Training Normalizing Flows

- **Goal:** Maximize the likelihood of the data under the model
- **Loss function:** Negative log-likelihood (to be minimized)

$$\mathcal{L}(x) = -\log p(x) = -\log p(z) - \log \left| \det \left(\frac{\partial f}{\partial x} \right) \right|$$

- ▶ $z = f(x)$: output of the flow
 - ▶ $p(z)$: usually a standard normal
 - ▶ The Jacobian term is computed efficiently via coupling layers
- Often report performance as Bits per Dimension:

$$\text{BPD} = -\frac{1}{D} \log_2 p(x)$$

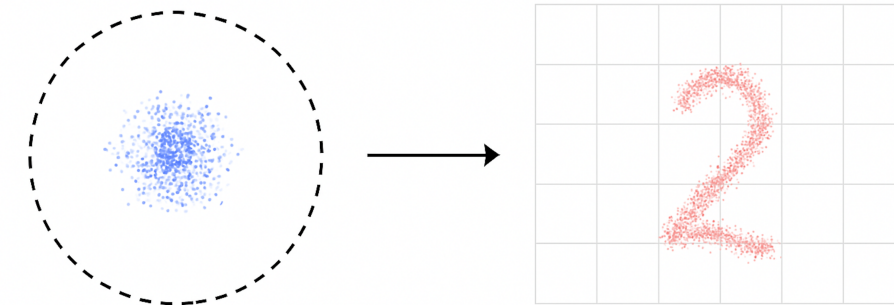
- ▶ Standardized unit of information (bits)
- ▶ Lower BPD $\rightarrow$ better model

Sampling with Flows

- Flows are **invertible**, so we can generate new data easily

- **Sampling Procedure:**

1. Sample $z \sim \mathcal{N}(0, I)$ from the base distribution
2. Compute $x = f^{-1}(z)$ using the inverse flow



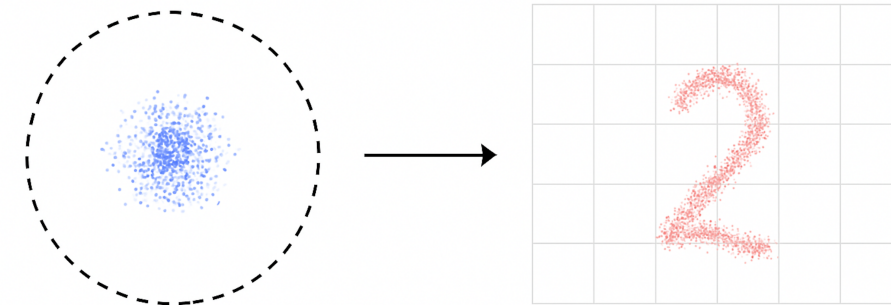
► **This gives a sample x from the learned data distribution**

Sampling with Flows

- Flows are **invertible**, so we can generate new data easily

- **Sampling Procedure:**

1. Sample $z \sim \mathcal{N}(0, I)$ from the base distribution
2. Compute $x = f^{-1}(z)$ using the inverse flow



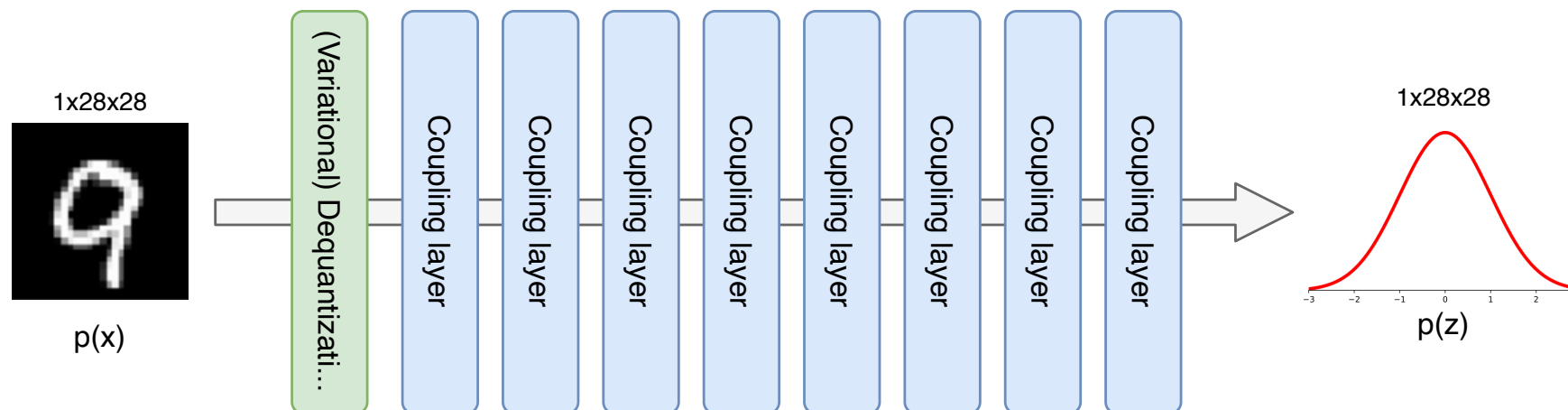
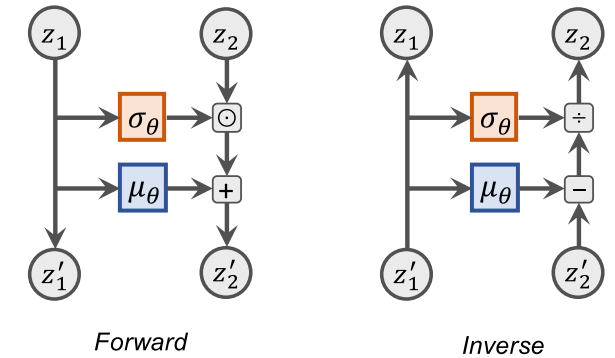
▶ **This gives a sample x from the learned data distribution**

- **Why it works?**

- ▶ Exact sampling — no approximation or noise
- ▶ Deterministic decoding: latent $z \rightarrow x$
- ▶ Enables interpolation, editing, etc.

Summary of the Vanilla Normalizing Flow Architecture

- **Input:** (Variationally) dequantized image data $x \in [0,1]^D$
- **Coupling layers:** invertible affine transformation:
 - ▶ Split input z into two parts
 - ➡ Efficient to invert and compute the log-determinant of the Jacobian
- **Output:** Latent variable z (e.g., Gaussian)



- **Training maximizes:**

$$\mathcal{L}(x) = -\log p(x) = -\log p(z) - \log \left| \det \left(\frac{\partial f}{\partial x} \right) \right|$$

The Challenge of High-Dimensional Data — Multi-Scale Flows

- Images (e.g., $28 \times 28 = 784$ dims) are **high-dimensional**
- **Learning a full transformation** over the **entire vector** is:
 - ▶ Computationally expensive
 - ▶ Statistically inefficient — not all pixels are equally informative

The Challenge of High-Dimensional Data — Multi-Scale Flows

- Images (e.g., $28 \times 28 = 784$ dims) are **high-dimensional**
- **Learning a full transformation** over the **entire vector** is:
 - ▶ Computationally expensive
 - ▶ Statistically inefficient — not all pixels are equally informative
- **A better design: Multi-Scale Flows:**
 - ▶ Instead of transforming everything at once:
 - ➡ We **gradually factor out** parts of the latent variables
 - ➡ Only **keep transforming what's harder to model**

The Challenge of High-Dimensional Data — Multi-Scale Flows

- Images (e.g., $28 \times 28 = 784$ dims) are **high-dimensional**
- **Learning a full transformation** over the **entire vector** is:
 - ▶ Computationally expensive
 - ▶ Statistically inefficient — not all pixels are equally informative
- **A better design: Multi-Scale Flows:**
 - ▶ Instead of transforming everything at once:
 - ➡ We **gradually factor out** parts of the latent variables
 - ➡ Only **keep transforming what's harder to model**

Key idea: Break down the problem hierarchically:
Remove simple parts early, focus depth on the rest

The Challenge of High-Dimensional Data — Multi-Scale Flows

- Images (e.g., $28 \times 28 = 784$ dims) are **high-dimensional**
- **Learning a full transformation** over the **entire vector** is:
 - ▶ Computationally expensive
 - ▶ Statistically inefficient — not all pixels are equally informative
- **A better design: Multi-Scale Flows:**
 - ▶ Instead of transforming everything at once:
 - ➡ We **gradually factor out** parts of the latent variables
 - ➡ Only **keep transforming what's harder to model**

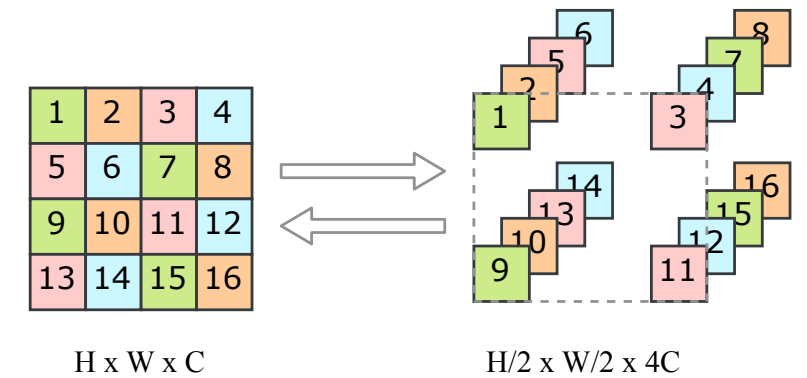
Key idea: Break down the problem hierarchically:
Remove simple parts early, focus depth on the rest

- **Outcome:**
 - ▶ Smaller models, better generalization.
 - ▶ Natural fit for hierarchical image structure (background vs. detail).

Squeeze and Split Operations

- **Squeeze: Reshape Spatial → Channel:**

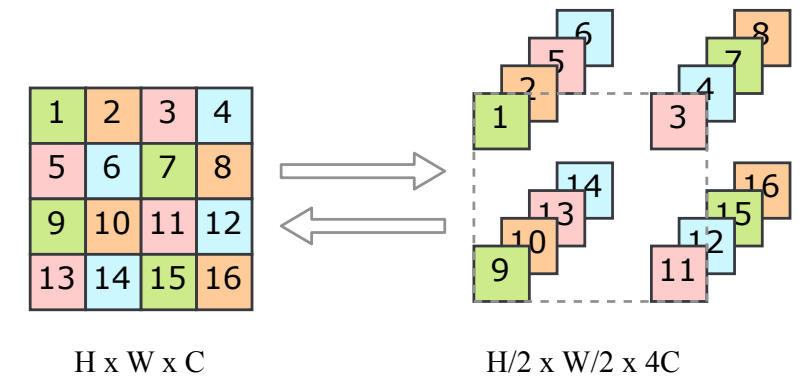
- ▶ Rearranges spatial patches into channels
- ▶ Reduces image size, increases dimensionality per pixel
- ▶ **Useful before applying coupling layers**



Squeeze and Split Operations

- **Squeeze: Reshape Spatial → Channel:**

- ▶ Rearranges spatial patches into channels
- ▶ Reduces image size, increases dimensionality per pixel
- ▶ **Useful before applying coupling layers**



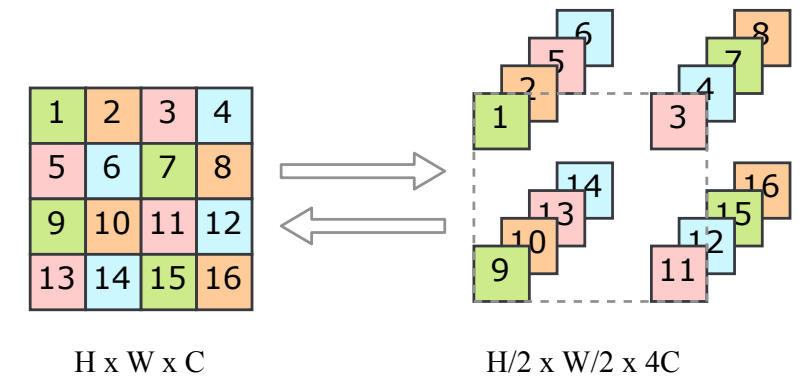
- **Split: Latent Factorization:**

- ▶ After some flow steps, **split off** a portion of z
- ▶ **Removed part** is **modeled with a prior** (e.g., $\mathcal{N}(0, I)$)
- ▶ Remaining part continues through deeper transformations

Squeeze and Split Operations

- **Squeeze: Reshape Spatial → Channel:**

- ▶ Rearranges spatial patches into channels
- ▶ Reduces image size, increases dimensionality per pixel
- ▶ **Useful before applying coupling layers**



- **Split: Latent Factorization:**

- ▶ After some flow steps, **split off** a portion of z
- ▶ **Removed part is modeled with a prior** (e.g., $\mathcal{N}(0, I)$)
- ▶ Remaining part continues through deeper transformations

- **Repeat the process:**

- ▶ Apply squeeze → flow → split multiple times.
- ▶ Creates a **multi-scale hierarchical representation**

Latent Variable Semantics

- **Deep Latents = Global Information**

- ▶ **Early layers (not split)** go through more transformations
- ▶ **Capture coarse structure**, e.g., digit identity or style
- ▶ Stored in the **deepest z-level**

Latent Variable Semantics

- **Deep Latents = Global Information**

- ▶ **Early layers (not split)** go through more transformations
- ▶ **Capture coarse structure**, e.g., digit identity or style
- ▶ Stored in the **deepest z-level**

- **Shallow Latents = Local Details**

- ▶ **Latents split early** are used to model finer details
- ▶ Help with **texture, edges, pixel precision**
- ▶ Closer to data space **$\mathbf{x}$** , **fewer transformations**

Latent Variable Semantics

- **Deep Latents = Global Information**

- ▶ **Early layers (not split)** go through more transformations
- ▶ **Capture coarse structure**, e.g., digit identity or style
- ▶ Stored in the **deepest z-level**

- **Shallow Latents = Local Details**

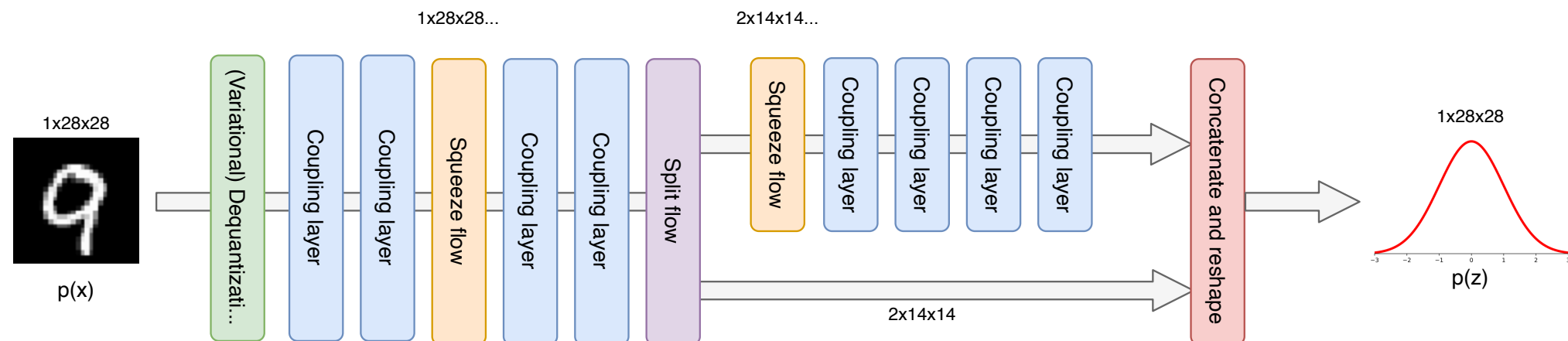
- ▶ **Latents split early** are used to model finer details
- ▶ Help with **texture, edges, pixel precision**
- ▶ Closer to data space $\mathbf{x}$, **fewer transformations**

- **Hierarchical Compression**

- ▶ Multi-scale flows act like an information funnel:
- ▶ **Deep latents = compact summary**
- ▶ **Shallow latents = high-frequency corrections**

Summary of the Multi-Scale Normalizing Flow Architecture

- Adds architectural **depth and efficiency** by progressively reshaping the data:
 - ▶ **Squeeze** operation: rearranges spatial structure into channels
 - ▶ **Split**: factor out part of the latent and apply prior early
 - ▶ Remaining part is passed through deeper flow layers

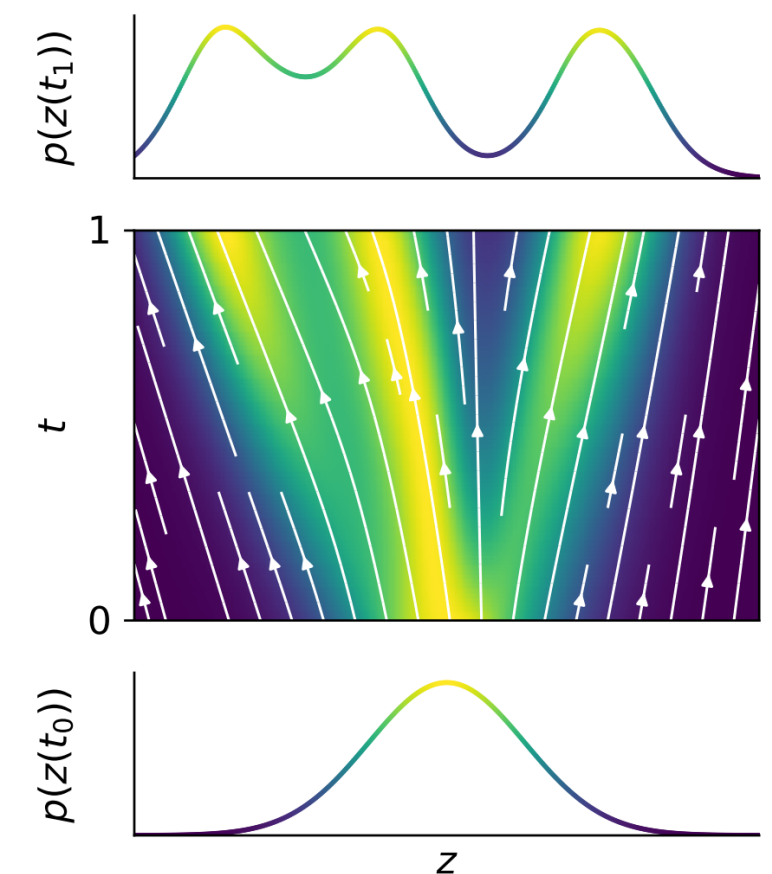
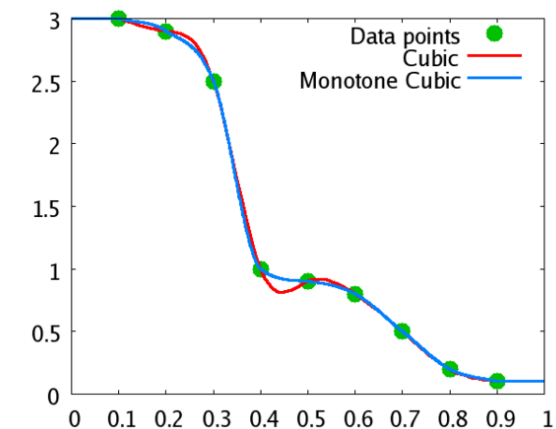


- **Advantages:**
- Captures **local structure** early, **global dependencies** later
- Allows **early likelihood contributions** via split latents
- Enables **scalable** training on high-dimensional inputs

Advanced Flow Architectures (I)

● Neural Spline Flows (NSF)

- ▶ Replace affine transforms with **monotonic splines**
- ▶ Learn more flexible 1D transformations
- ▶ Keeps invertibility and tractable Jacobian
- ▶ **Use when:** affine layers are too rigid to model sharp or multimodal densities

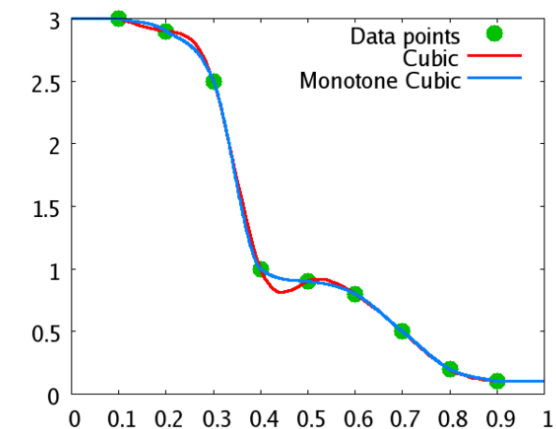


[W. Grathwohl, "FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models", ICLR 2019](#)

Advanced Flow Architectures (I)

● Neural Spline Flows (NSF)

- ▶ Replace affine transforms with **monotonic splines**
- ▶ Learn more flexible 1D transformations
- ▶ Keeps invertibility and tractable Jacobian
- ▶ **Use when:** affine layers are too rigid to model sharp or multimodal densities

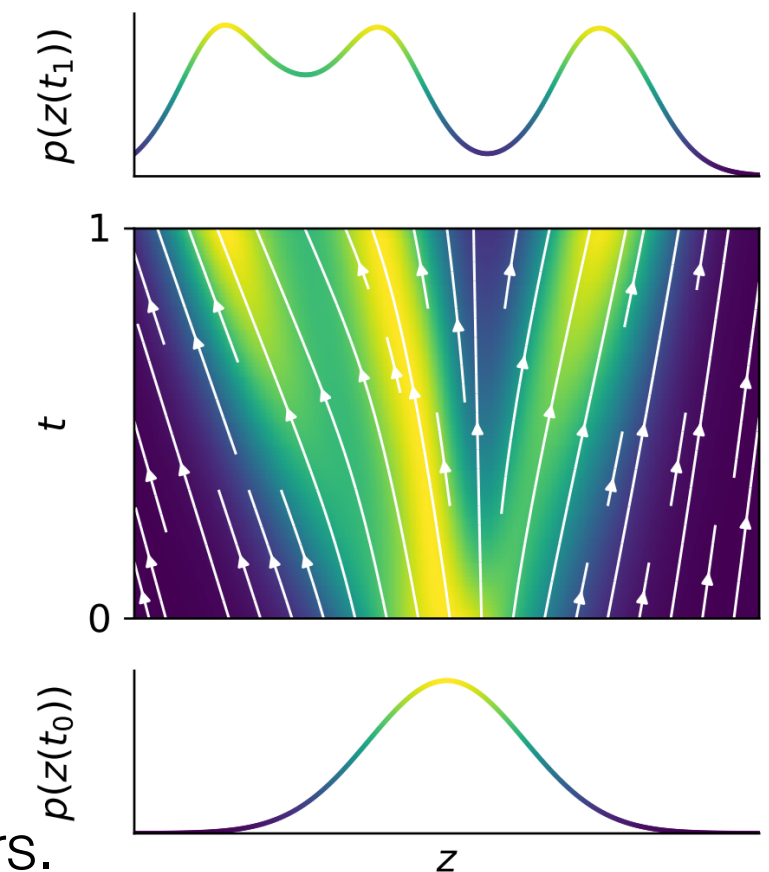


● Continuous Normalizing Flows (CNF)

- ▶ Replace discrete layers with **differential equations**
- ▶ Learn a transformation via an ODE:

$$\frac{dz(t)}{dt} = f_{\theta}(z(t), t)$$

- ▶ Use the ODE solver and Neural ODE machinery
- ▶ **Key benefit:** flexible, adaptive transformations — fewer layers.

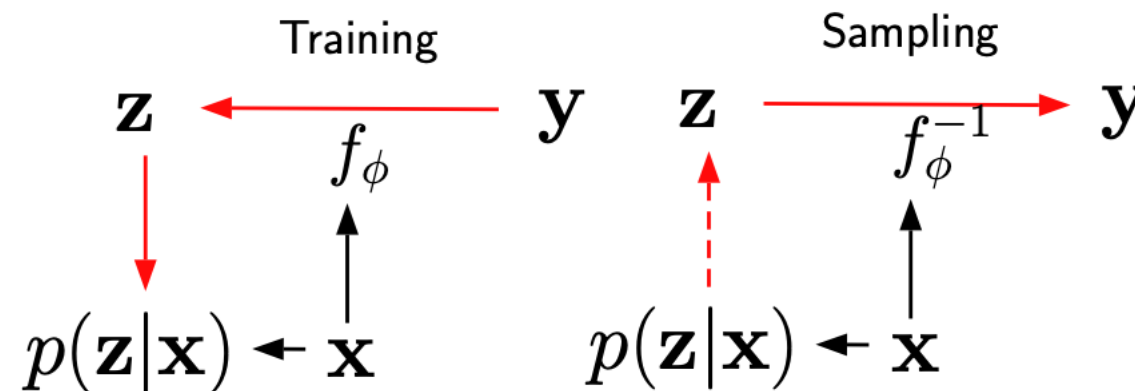


[W. Grathwohl, "FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models", ICLR 2019](#)

Advanced Flow Architectures (II)

● Conditional Flows

- ▶ Learn $p(y | x)$ instead of $p(y)$
- ▶ Condition the transformation on side information (e.g., class, text)
- ▶ Enables powerful conditional generation and inference
- ▶ **Applications:** image-to-image translation, molecular property control



[C. Winkler et al., "Learning Likelihoods with Conditional Normalizing Flows", ICLR 2020](#)

Applications Beyond Images

- **Molecules & Chemistry**

- ▶ Learn invertible mappings from latent space to molecules
- ▶ Enable molecular design with controlled properties

- **Time Series & Forecasting**

- ▶ Use flows for density estimation of sequential data
- ▶ Combine with RNNs or attention for autoregressive modeling
- ▶ **Applications:** weather, finance, speech

- **Graphs**

- ▶ Model graph structures via permutation-invariant flows
- ▶ Useful in molecule generation or social network modeling

- **Probabilistic Inference**

- Use flows to approximate complex posteriors in Bayesian models
- Plug into variational inference or MCMC pipelines

[Y. Saleh et al., “Computing Excited States of Molecules Using Normalizing Flows”, J. Chem. Theory Comput., vol. 21, no. 10, pp. 5221–5229, 2025](#)

[K. Rasul et al., “Multivariate Probabilistic Time Series Forecasting via Conditioned Normalizing Flows”, ICLR 2021 \(Spotlight\)](#)

[J. Liu et al., “Graph Normalizing Flows”, NeurIPS 2019](#)

[D. J. Rezende et al., “Variational Inference with Normalizing Flows”, ICML 2015](#)

References

- Bishop, C. M., Bishop, H. & Cham, S. (ed.) (2023). Deep Learning - Foundations and Concepts. ISBN: 978-3-031-45468-4
- Tomczak, Jakub M. Deep Generative Modeling. 2nd ed. Cham: Springer, 2024. <https://doi.org/10.1007/978-3-031-64087-2>
- Lippe, Phillip. “Tutorial 11: Normalizing Flows for Image Modeling.” UVA Deep Learning Course Notebooks. Accessed July 3, 2025. https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/tutorial11/NF_image_modeling.html

Additional References

- Dinh, L., Sohl-Dickstein, J., and Bengio, S. (2017). “Density estimation using Real NVP,” In: 5th International Conference on Learning Representations, ICLR 2017. [Link](#)
- Kingma, D. P., and Dhariwal, P. (2018). “Glow: Generative Flow with Invertible 1x1 Convolutions,” In: Advances in Neural Information Processing Systems, vol. 31, pp. 10215–10224. [Link](#)
- Ho, J., Chen, X., Srinivas, A., Duan, Y., and Abbeel, P. (2019). “Flow++: Improving Flow-Based Generative Models with Variational Dequantization and Architecture Design,” in Proceedings of the 36th International Conference on Machine Learning, vol. 97, pp. 2722–2730. [Link](#)
- Durkan, C., Bekasov, A., Murray, I., and Papamakarios, G. (2019). “Neural Spline Flows,” In: Advances in Neural Information Processing Systems, pp. 7509–7520. [Link](#)
- Hoogetboom, E., Cohen, T. S., and Tomczak, J. M. (2020). “Learning Discrete Distributions by Dequantization,” arXiv preprint arXiv2001.11235v1. [Link](#)
- Lippe, P., and Gavves, E. (2021). “Categorical Normalizing Flows via Continuous Transformations,” In: International Conference on Learning Representations, ICLR 2021. [Link](#)